

Run Together

Le réseau social collaboratif pour vos activités sportives

Projet encadré par Amélie Gheerbrant

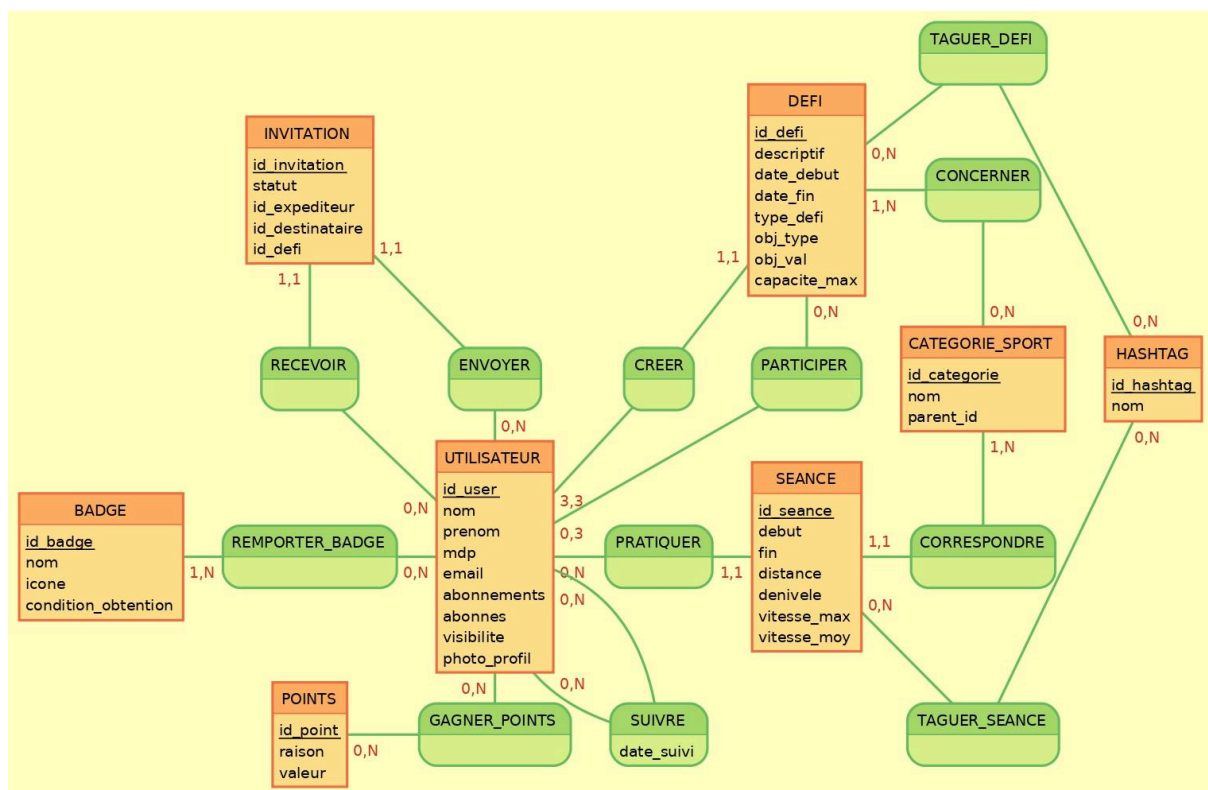
Unité d'enseignement : Bases de données

1 - Introduction

RunTogether est une application collaborative centrée sur les activités sportives d'endurance. Elle permet à ses utilisateurs d'enregistrer leurs séances, de créer, de rejoindre des défis individuels ou collectifs et d'interagir avec d'autres utilisateurs grâce à un système de suivi, de buddies et d'invitations.

Ce rapport présente les choix de modélisation effectués, le schéma relationnel, les requêtes implémentées, ainsi que l'indice de recommandation de défis.

2 - Modèle entités-associations



- L'association SUIVRE est réflexive : un utilisateur en suit un autre. Elle n'est pas symétrique car les utilisateurs ne sont pas obligés de se suivre entre eux. Deux utilisateurs sont buddies si la relation est réciproque.
- Pour modéliser la hiérarchie des catégories sportives nous utilisons la clé parent_id qui est elle aussi une relation réflexive car une catégorie pointe vers une autre catégorie.

3. Schéma relationnel

- utilisateur (**id_user**, nom, prenom, mdp, email, abonnements, abonnées, visibilité, photo_profil)
- seance (**id_seance**, debut, fin, distance, denivele, vitesse_max, vitesse_moy)
- categorie_sport (**id_categorie**, nom, #parent_id)
- defi (**id_defi**, descriptif, date_debut, date_fin, type_defi, obj_type, obj_val, capacite_max)
- hashtag (**id_hashtag**, nom)
- badge (**id_badge**, nom, icone, condition_obtention)
- invitation (**id_invitation**, statut, #id_expediteur, #id_destinataire, #id_defi)
- points (**id_point**, raison, valeur)
- correspond (#id_seance, #id_categorie)
- pratiquer (#id_user, #id_seance)
- taguer_seance (#id_seance, #id_hashtag)
- concerner (#id_defi, #id_categorie)
- creer (#id_user, #id_defi)
- participer (#id_user, #id_hashtag)
- taguer_defi (#id_defi, #id_hashtag)
- remporter_badge (#id_user, #id_badge)
- gagner_points (#id_user, #id_point)
- suivre (#id_follower, #id_followed, date_suivi)

Gestion des contraintes

- CHECK : afin d'avoir des données cohérentes, nous vérifions que certaines valeurs ne peuvent pas être négatives telles que la distance ou la vitesse maximale. Nous faisons aussi en sorte que la fin d'une séance (type TIMESTAMP) soit supérieure à son début.
- Contrainte de non nullité : on retrouve beaucoup de champs obligatoires marqués avec la syntaxe NOT NULL comme le nom d'une catégorie sportive ou le mot de passe d'un utilisateur.
- Contrainte d'unicité : certains champs sont marqués comme UNIQUE comme l'adresse email pour empêcher qu'un utilisateur crée plusieurs comptes, ou le nom d'un hashtag pour éviter les doublons.
- Contrainte de nettoyage de la table : le mot-clé CASCADE permet de supprimer automatiquement les dépendances. Nous supprimons d'abord les tables d'association puis les tables racines car à cause des clés étrangères, on ne peut pas supprimer une table parent avant son enfant.

Il existe également des contraintes gérées par l'application. « Un utilisateur ne peut créer ni participer à plus de trois défis par semaine ». Cela nécessite une vérification par le code avant d'insérer dans la table. « Le créateur d'un défi est automatiquement un participant ». Lorsqu'un défi est créé, il faut simultanément insérer id_user dans la table DEFI et l'association PARTICIPER.

4. Requêtes SQL

1. 3 tables : Les utilisateurs et le nombre de défis collectifs auxquels ils sont en train de participer
2. Auto-jointure : Récupère les buddies
3. Auto-jointure avec noms : Identique à la requête précédente mais affiche l'identité au lieu de l'id
4. Sous-requête corrélée : Utilisateurs qui ont participé à un défi avec un buddy
5. Sous-requête dans le FROM : Utilisateurs ayant envoyé plus de 5 invitations
6. Sous-requête dans le FROM : Utilisateurs ayant participé à au moins 2 défis individuels
7. Sous-requête dans le FROM : Nombre de défis créés pour chaque catégorie de sport
8. Sous-requête dans le WHERE : Utilisateurs qui ont gagné le badge "feu aux fesses"
9. Agrégats nécessitant GROUP BY et HAVING : Utilisateurs ayant fait au moins 2 séances
10. Agrégats nécessitant GROUP BY et HAVING : Utilisateurs dont la vitesse moyenne sur toutes les séances est d'au moins 8
11. Deux agrégats : Utilisateurs qui ont créé les défis les plus populaires
12. Jointure externe (LEFT JOIN) : Tous les hashtags avec les défis associés
13. Jointure externe (RIGHT JOIN) : Tous les défis avec le nombre de participants
14. Condition de totalité (Agrégation) : Utilisateurs qui ont pratiqué au moins une séance dans toutes les catégories sportives existantes
15. Condition de totalité (Sous-requête corrélée) : Identique à la précédente
16. NULL (Résultats différents) : Nombre d'utilisateurs qui ont une photo de profil et nombre total d'utilisateurs
17. NULL (Mêmes résultats) : Nombre d'utilisateurs qui ont une photo de profil
18. Requête récursive : Tous les utilisateurs accessibles via les abonnements de l'utilisateur 1
19. Fenêtrage (DENSE_RANK) : Classe les utilisateurs par distances totale parcourue
20. Fenêtrage : Calcule la distance cumulée parcourue par chaque utilisateur

5. Indice de recommandation

$$\text{Indice} = (\text{Niveau} * 0,4 + \text{Habitudes} * 0,3 + \text{Social} * 0,3)$$

Le niveau mesure si le défi est adapté à l'utilisateur. Pour mesurer le niveau d'un utilisateur, on calcule, sur toutes ses séances, la distance moyenne parcourue et on la compare à tous les défis existants pour filtrer quels défis sont au niveau (ni trop facile, ni trop difficile) de l'utilisateur.

Les habitudes mesurent à quelle fréquence l'utilisateur pratique le sport du défi. Il est calculé en divisant le nombre de séances qui correspondent au sport, divisé par le nombre total de séances de l'utilisateur.

Enfin le social mesure la proportion d'abonnements de l'utilisateur qui participent déjà au défi : plus il y a de personnes suivies par l'utilisateur qui participent à un défi donné, plus celui-ci va lui être recommandé.

Plus l'indice de recommandation sera élevé, plus le défi sera recommandé à l'utilisateur car il sera considéré comme intéressant pour celui-ci.